

深度学习中的embedding

 机器学习 专栏收录该内容

15 篇文章

摘要 本文介绍了深度学习中处理分类输入数据，特别是embedding的应用。embedding能够将稀疏的one-hot编码转换为低维向量，保留语义关系网络大小和向量间关系的问题。文章讨论了embedding的获取方法，如词向量模型Word2vec，以及在TensorFlow中使用embedding_lookup。此外，还介绍ding作为更大模型一部分进行训练的实践案例。

摘要生成于 [C知道](#)，由 DeepSeek-R1 满血版支持， [前往体验](#) >

整理翻译自google developer的机器学习入门课程，介绍了embedding的应用方式和如何计算embedding，后面还配有通过tensorflow DNN训练embedding理解。

分类输入数据(Categorical Input Data)

分类数据是指表示来自有限选择集的一个或多个离散项的输入特征。例如，它可以是用户观看的电影集，文档中的单词集或人的职业。

分类数据通过稀疏张量(sparse tensors)表示最有效，稀疏张量是具有非常少的非零元素的张量。例如，如果我们正在构建电影推荐模型，我们可以为电影分配一个唯一的ID，然后通过用户观看过的电影的稀疏张量来表示每个用户，如图3所示。



Figure 3. Data for our movie recommendation problem.

图3中的矩阵的每一行是捕获用户的电影观看历史的示例，被表示为稀疏张量，因为每个用户仅观看所有可能电影的一小部分。最后一行对应于稀疏张量[1,3,999999]，使用电影图标上方显示的词索引。

同样，人们可以将单词、句子和文档表示为稀疏向量，其中词汇表中的每个单词起到与我们的推荐示例中的电影类似的作用。

为了在机器学习系统中使用这样的表示，我们需要一种方法来将每个稀疏向量表示为数字向量，以便语义上相似的项（电影或单词）在向量空间中具有离。但是，你如何将一个单词表示为数字向量？

最简单的方法是为词汇表中的每个单词定义一个带有节点的巨型输入层，或者至少为数据中出现的每个单词定义一个节点。如果数据中出现500,000个可以表示为长度为500,000向量的单词，并将每个单词分配给向量中的一个位置(slot)。

如果将“horse”分配给索引1247，然后将“horse”提供给网络，则可以将1复制到第1247个输入节点，将0复制到所有其余节点。这种表示称为one-hot编码，一个索引具有非零值。

更典型地，向量可能包含更大块文本中的单词计数。这被称为“词袋 (bag of words)”表示。在一个词袋向量中，500,000个节点中的有几个将具有非零值。但是，无论怎么确定非零值，一个节点一个单词都会得到非常稀疏的输入向量—非常大的向量，只具有相对较少的非零值。稀疏表示存在一些问题，这使模型难以有效学习。主要问题是构造的ont-hot vector太大以及vector之间距离刻画问题。

网络大小

巨大的输入向量意味着神经网络的超大数量的权重。如果你的词汇表中有M个单词，并且输入上的网络的第一层中有N个节点，则您需要使用MxN个权层。大量的权重会导致进一步的问题：

1. 数据量：模型中的权重越多，有效训练需要的数据就越多。
2. 计算量：权重越多，训练和使用模型所需的计算量就越多。很容易超出硬件的能力。

向量间缺少有意义的关系

如果将RGB通道的像素值提供给图像分类器，那么谈论“相近”值是有意义的。略带红色的蓝色接近纯蓝色，无论是在语义上还是在向量之间的几何距离。对于索引1247的1的向量表示的“马”，与索引50,430为1的向量表示的“羚羊”的关系不比索引238为1表示的向量“电视”的更接近。

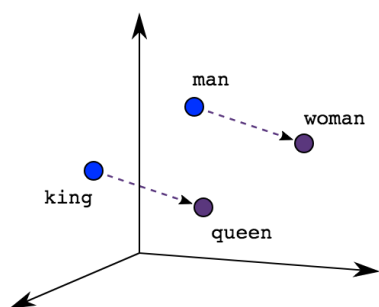
解决方案：embedding

这些问题的解决方案是使用embedding，**embedding**将大型稀疏向量转换为保留语义关系的低维空间。我们将在本单元的以下部分中直观地、概念上探索embedding。

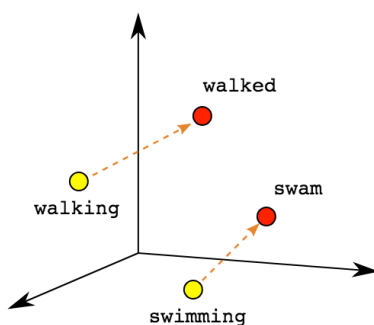
embedding

您可以通过将高维数据映射到较低维空间来解决稀疏输入数据的核心问题。

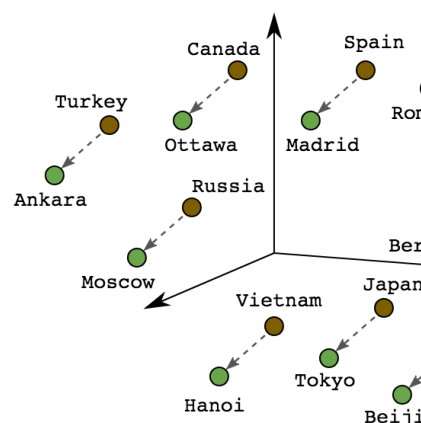
即使是一个很小的多维空间也可以自由地将语义相似的item组合在一起，并使不同的item隔开。向量空间中的位置（距离和方向）可以把语义编码到embedding中。例如，以下真实的embedding可视化显示了捕获的语义关系，如国家与其首都之间的几何关系：



Male-Female



Verb Tense



Country-Capital

这种有意义的空间为机器学习系统提供了检测可能有助于学习任务的机会。

Shrinking the network(收缩网络)

虽然我们需要足够的维度来编码丰富的语义关系，但我们想要一个足够小的embedding空间，以便我们能够更快地训练我们的系统。一个有用的embedding数百个维度的量级，这可能比自然语言任务的词汇量小几个数量级。

Embeddings as lookup tables(embedding看作查表)

embedding是一种矩阵，其中每列是与词汇表中的item对应的向量。要获取单个词汇item的稠密向量，就检索与该item对应的列。可以参照tensorflow的tf.nn.embedding_lookup函数。

但你怎么转换一堆稀疏的词袋(bag of words)向量？要获得表示多个词汇item（例如句子或段落中的所有单词）的稀疏向量的稠密向量，可以检索每个embedding，然后将它们相加。

如果稀疏向量包含词汇项的计数，则可以将每个embedding乘以其相应项的计数，然后再相加。

Embedding lookup as matrix multiplication(查找看作矩阵乘法)

我们刚才描述的查找、乘法和加法过程等同于矩阵乘法。给定 $1 \times N$ 稀疏表示S和 $N \times M$ 的embedding表E，矩阵乘法 $S \times E$ 给出 $1 \times M$ 稠密向量。但是你怎么在第一时间得到E？我们将在下一节中介绍如何获取embedding。

Obtaining Embeddings(获取embedding)

有许多方法可以实现embedding，包括在Google创建的最先进的算法。

Standard Dimensionality Reduction Techniques(标准降维技术)

有许多用于捕获低维空间中的高维空间的重要结构的现有数学技术。理论上，这些技术中的任何一种都可用于为机器学习系统创建embedding。

例如，主成分分析（PCA）已用于创建单词embedding。给定一组像词袋向量一样的实例，PCA试图找到可以折叠成单个维度的高度相关的维度。

Word2vec

Word2vec是Google发明的用于训练单词embedding的算法。 Word2vec依赖于分布假设(distributional hypothesis)来将语义相似的词映射到几何上相近向量。

word2vec参考<https://www.tensorflow.org/tutorials/word2vec/index.html>

Training an Embedding as Part of a Larger Model(当作训练网络的一部分)

您还可以通过作为目标任务的神经网络的一部分来学习embedding。 这种方法可以为特定系统定制良好的embedding，但可能需要比单独训练embedd时间。

通常，当有稀疏数据（或您想要作embedding的稠密数据）时，您可以创建一个embedding单元，它只是一种特殊类型的大小为d的隐藏单元。 该emb与任何其他特征和隐藏层组合。 与任何DNN一样，最后一层将是优化的损失函数。 例如，假设我们正在执行协同过滤，其目标是根据其他用户的兴趣兴趣。 我们可以将此模型建模为监督学习问题，方法是随机拨出（或保留）用户已观看的少量电影作为正label，然后优化softmax损失。

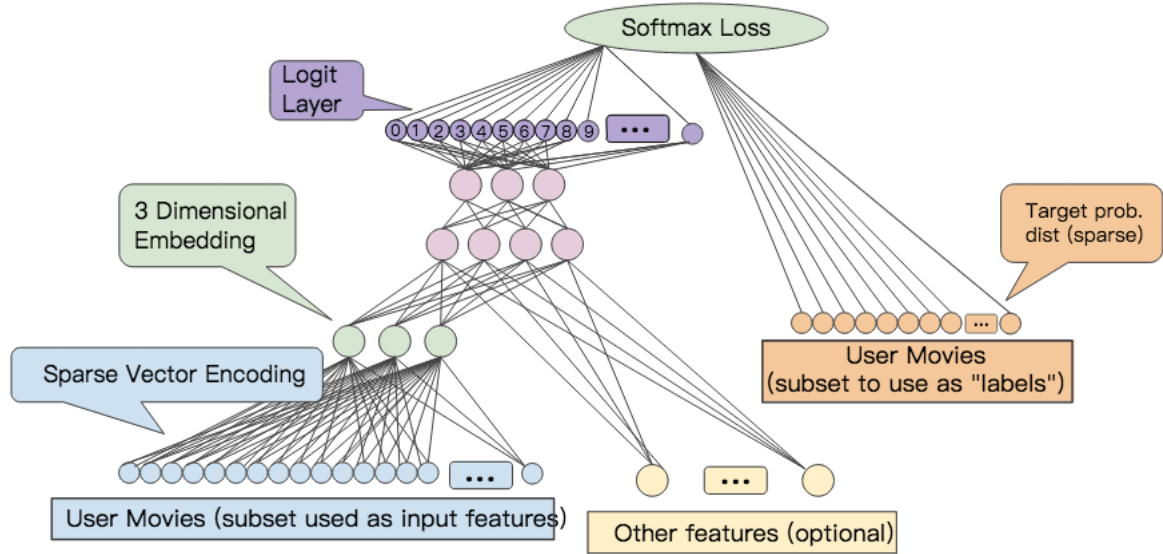


Figure 5. A sample DNN architecture for learning movie embeddings from collaborative filtering data.

另外一个例子是，如果您想为房地产广告中的字词创建一个embedding层，作为预测房价的DNN的一部分，那么可以使用训练数据中已知的房屋销售价格来优化L2损失函数。

当学习d维embedding时，每个item被映射到d维空间中的点，使得相似的item在该空间中是相近的。 图6有助于说明embedding层中学习的权重与几何关系。 输入节点和d维embedding层中的节点之间的边权重对应于每个d轴的坐标值。

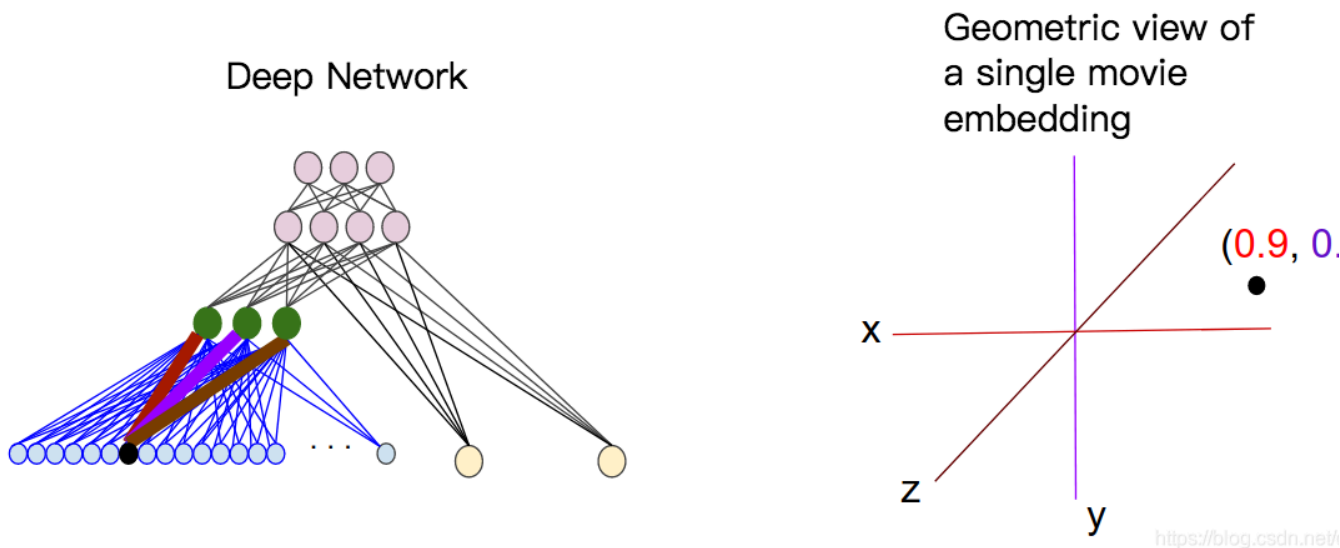


Figure 6. A geometric view of the embedding layer weights.

实践

google通过Colaboratory提供了notebook形式交互学习embedding的训练，如果可以访问国外网站，可以直接去[该地址](#)。

目标：通过电影评论的数据集 [ACL 2011](#) [IMDB dataset](#)训练一个语义分析模型来预测影评是喜欢(label=1)还是不喜欢(label=0)。

关于数据集：

数据集是电影的影评和评分，每个电影有个唯一id和对应评分，文件形式：id_score.txt，内容是影评。

原始数据集包括了train和test两个目录，train下面的neg目录表示负样本，其中评分score<=4，label为0；pos目录表示正样本，其中评分score>=7，lat其中用的数据已经转换为tfrecord格式数据了。

特征工程：

通过tf.feature_column.categorical_column_with_vocabulary_list函数把影评的terms（影评的单词）转换为feature vector。本示例是为了练习，因此采(vocabulary)方式，只选取小部分词汇表，50个。（在实践中经常采用特征hash方式）

pythonAI生成项目登录

```
1 informative_terms = ("bad", "great", "best", "worst", "fun", "beautiful",
2                       "excellent", "poor", "boring", "awful", "terrible",
3                       "definitely", "perfect", "liked", "worse", "waste",
4                       "entertaining", "loved", "unfortunately", "amazing",
5                       "enjoyed", "favorite", "horrible", "brilliant", "highly",
6                       "simple", "annoying", "today", "hilarious", "enjoyable",
7                       "dull", "fantastic", "poorly", "fails", "disappointing",
8                       "disappointment", "not", "him", "her", "good", "time",
9                       "?", ".", "!", "movie", "film", "action", "comedy",
10                      "drama", "family")
```

收起

上面的50个词汇表可以看作是size为50的feature vector，如果影评中出现改词，则该位置为1，否则为0，影评中没有出现的丢弃。（当然，上面的feaure vector可以直接用线性模型或DNN来训练）

embedding

本示例主要为了练习使用embedding，上面的feature vector就是稀疏的分类数据，可以转换为低纬的embedding（dense），使用的是tf.feature_column.embedding_column方法，

把相关python code贴下来：

pythonAI生成项目登录

```
1 from __future__ import print_function
2
3 import collections
4 import io
5 import math
6
7 import numpy as np
8 import pandas as pd
9 import tensorflow as tf
10
11 # 下载已经转换为tfrecord形式的数据集
12 tf.logging.set_verbosity(tf.logging.ERROR)
13 train_url = 'https://download.mlcc.google.com/mledu-datasets/sparse-data-embedding/train.tfrecord'
14 train_path = tf.keras.utils.get_file(train_url.split('/')[-1], train_url)
15 test_url = 'https://download.mlcc.google.com/mledu-datasets/sparse-data-embedding/test.tfrecord'
16 test_path = tf.keras.utils.get_file(test_url.split('/')[-1], test_url)
17
18 def _parse_function(record):
19     """Extracts features and labels.
20     Args:
21         record: File path to a TFRecord file
22     Returns:
23         A `tuple` `(labels, features)`:
24             features: A dict of tensors representing the features
25             labels: A tensor with the corresponding labels.
26     """
27     features = {
28         "terms": tf.VarLenFeature(dtype=tf.string), # terms are strings of varying lengths
29         "labels": tf.FixedLenFeature(shape=[1], dtype=tf.float32) # labels are 0 or 1
30     }
31
32     parsed_features = tf.parse_single_example(record, features)
33
34     terms = parsed_features['terms'].values
35     labels = parsed_features['labels']
36
37     return {'terms':terms}, labels
38 # Create an input_fn that parses the tf.Examples from the given files,
39
```

```

40 # and split them into features and targets.
41 def _input_fn(input_filenames, num_epochs=None, shuffle=True):
42
43     # Same code as above; create a dataset and map features and labels.
44     ds = tf.data.TFRecordDataset(input_filenames)
45     ds = ds.map(_parse_function)
46
47     if shuffle:
48         ds = ds.shuffle(10000)
49
50     # Our feature data is variable-length, so we pad and batch
51     # each field of the dataset structure to whatever size is necessary.
52     ds = ds.padded_batch(25, ds.output_shapes)
53
54     ds = ds.repeat(num_epochs)
55
56
57     # Return the next batch of data.
58     features, labels = ds.make_one_shot_iterator().get_next()
59     return features, labels
60
61 # 50 informative terms that compose our model vocabulary
62 informative_terms = ("bad", "great", "best", "worst", "fun", "beautiful",
63                     "excellent", "poor", "boring", "awful", "terrible",
64                     "definitely", "perfect", "liked", "worse", "waste",
65                     "entertaining", "loved", "unfortunately", "amazing",
66                     "enjoyed", "favorite", "horrible", "brilliant", "highly",
67                     "simple", "annoying", "today", "hilarious", "enjoyable",
68                     "dull", "fantastic", "poorly", "fails", "disappointing",
69                     "disappointment", "not", "him", "her", "good", "time",
70                     "?", ".", "!", "movie", "film", "action", "comedy",
71                     "drama", "family")
72
73
74 terms_feature_column = tf.feature_column.categorical_column_with_vocabulary_list(key="terms", vocabulary_list=informative_terms)
75 terms_embedding_column = tf.feature_column.embedding_column(terms_feature_column, dimension=2)
76 feature_columns = [ terms_embedding_column ]
77
78 my_optimizer = tf.train.AdagradOptimizer(learning_rate=0.1)
79 my_optimizer = tf.contrib.estimator.clip_gradients_by_norm(my_optimizer, 5.0)
80
81 classifier = tf.estimator.DNNClassifier(
82     feature_columns=feature_columns,
83     hidden_units=[20,20],
84     optimizer=my_optimizer
85 )
86
87 classifier.train(
88     input_fn=lambda: _input_fn([train_path]),
89     steps=1000)
90
91 evaluation_metrics = classifier.evaluate(
92     input_fn=lambda: _input_fn([train_path]),
93     steps=1000)
94 print("Training set metrics:")
95 for m in evaluation_metrics:
96     print(m, evaluation_metrics[m])
97 print("---")
98
99 evaluation_metrics = classifier.evaluate(
100     input_fn=lambda: _input_fn([test_path]),
101     steps=1000)
102
103 print("Test set metrics:")
104 for m in evaluation_metrics:
105     print(m, evaluation_metrics[m])
106 print("---")

```

收起 ^

上面这段代码运行结果如下：

```

1 Training set metrics:
2 accuracy 0.78008
3 accuracy_baseline 0.5
4 auc 0.8669316
5 auc_precision_recall 0.85548073
6 average_loss 0.46098813
7 label/mean 0.5
8 loss 11.524703
9 precision 0.805978
10 prediction/mean 0.46057376
11 recall 0.73776
12 global_step 1000
13 ---
14 Test set metrics:
15 accuracy 0.77732
16 accuracy_baseline 0.5
17 auc 0.86572266
18 auc_precision_recall 0.85332245
19 average_loss 0.46291515
20 label/mean 0.5
21 loss 11.572879
22 precision 0.8051228
23 prediction/mean 0.4597779
24 recall 0.73176
25 global_step 1000
26 ---
27

```

收起 ^

查看embedding效果

上面的训练完成后，我们可以看看训练的embedding效果，其中embedding_matrix就是训练出来的embedding矩阵。

新增如下code

python

AI生成项目

登录

```

1 import matplotlib.pyplot as plt
2
3 embedding_matrix = classifier.get_variable_value('dnn/input_from_feature_columns/input_layer/terms_embedding/embedding_
4
5 for term_index in range(len(informative_terms)):
6     # Create a one-hot encoding for our term. It has 0s everywhere, except for
7     # a single 1 in the coordinate that corresponds to that term.
8     term_vector = np.zeros(len(informative_terms))
9     term_vector[term_index] = 1
10    # We'll now project that one-hot vector into the embedding space.
11    embedding_xy = np.matmul(term_vector, embedding_matrix)
12    plt.text(embedding_xy[0],
13             embedding_xy[1],
14             informative_terms[term_index])
15
16 # Do a little setup to make sure the plot displays nicely.
17 plt.rcParams["figure.figsize"] = (15, 15)
18 plt.xlim(1.2 * embedding_matrix.min(), 1.2 * embedding_matrix.max())
19 plt.ylim(1.2 * embedding_matrix.min(), 1.2 * embedding_matrix.max())
20 plt.show()

```

收起 ^

整理翻译自Google开发者机器学习课程